

TABLE OF CONTENTS

Flutter App Architecture: The Domain Model	1
Flutter Layered Architecture	1
What is a Domain Model?	3
The Domain Layer	5
Business logic in the Model classes	7
Testing the Business Logic inside our Models	8
Conclusion	9
Flutter Foundations Course Now Available	10
Flutter Foundations Course	10
Want More?	11
Flutter In Production	11
Flutter & Firebase Masterclass	11
The Complete Dart Developer Guide	11
Flutter Animations Masterclass	11



Flutter App Architecture: The Domain Model

#dart

#flutter

#app-architecture



Andrea Bizzotto

Feb 9, 2022 · 9 min read

Have you ever mixed up your **UI**, **business logic** and **networking** code into one messy bundle of spaghetti code?

I know I did. 🙄

After all, real-world app development is hard.

Books such as **Domain-Driven Design** (DDD) have been written to help us develop complex software projects.

And at the heart of DDD lies the **model**, which captures **important knowledge** and **concepts** that are needed to solve the problem at hand. And having a good **domain model** can make all the difference between the success or failure of a software project.

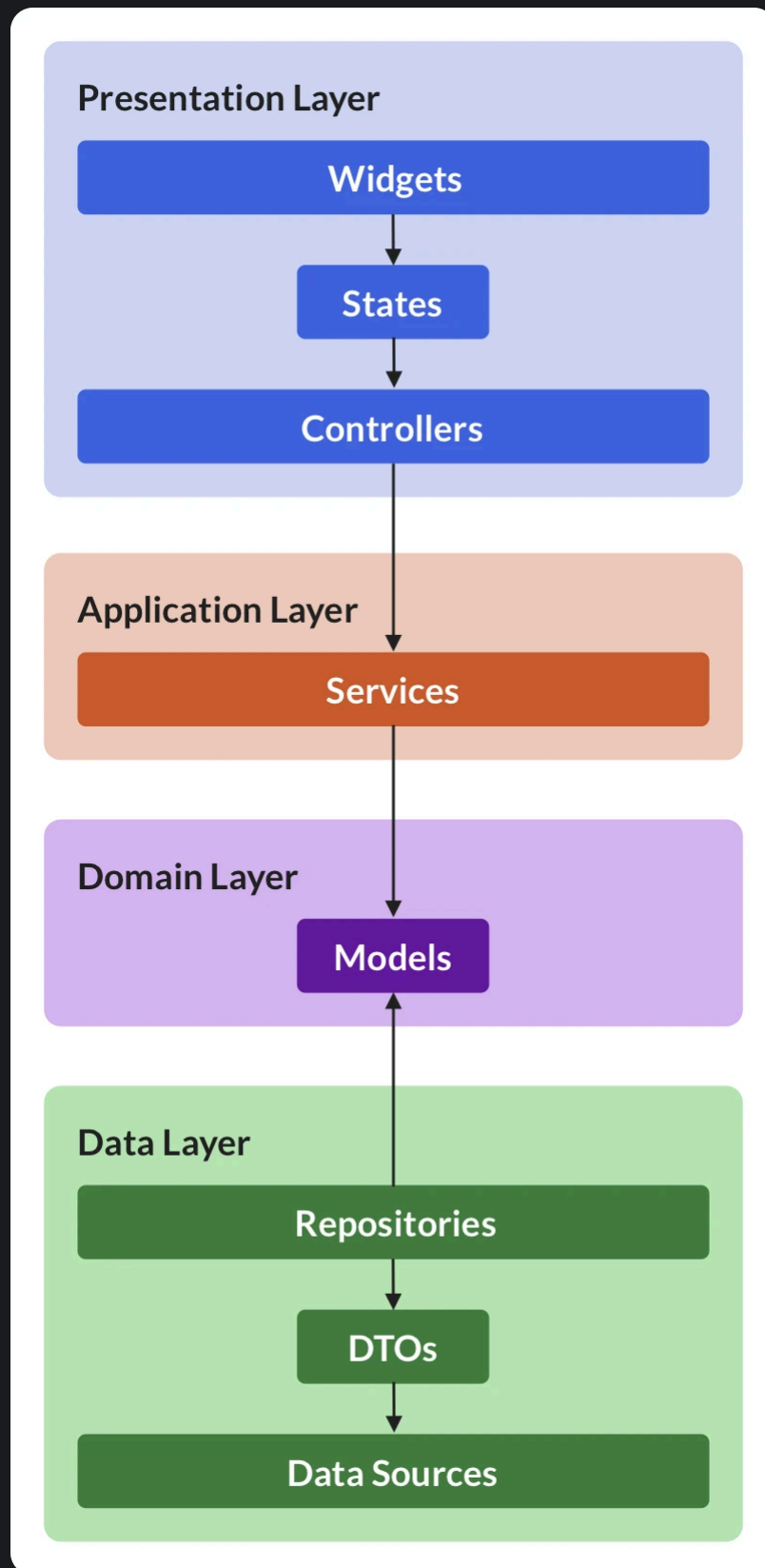
Models are very important but they cannot live in isolation. Even the simplest of apps requires some UI (what the user sees and interacts with) and needs to communicate with external APIs to show some meaningful information.

Flutter Layered Architecture

In this context, it's often valuable to adopt a **layered** architecture, introducing a clear **separation of concerns** between different parts of the system. And this makes our code easier to read, maintain and test.

Broadly speaking, four different layers are normally identified:

- presentation layer
- application layer
- domain layer
- data layer



Flutter App Architecture using data, domain, application, and presentation layers.

The **data layer** sits at the bottom and contains the repositories that are used to talk to external data sources.

Just above it, we find the **domain** and **application** layers. These layers are very important as they hold all the **models** and **business logic** of our app. For more info, read the other articles in this series:

- [Flutter App Architecture with Riverpod: An Introduction](#)
- [Flutter Project Structure: Feature-first or Layer-first?](#)
- [Flutter App Architecture: The Repository Pattern](#)
- [Flutter App Architecture: The Presentation Layer](#)
- [Flutter App Architecture: The Application Layer](#)

In this article, we'll focus on the **domain layer**, using an eCommerce application as a practical example. As part of this, we will learn:

- what is a domain model
- how to define entities and represent them as data classes in Dart
- how to add business logic to our model classes
- how to write unit tests for that business logic

Ready? Let's go!

What is a Domain Model?

Wikipedia defines the domain model like this:

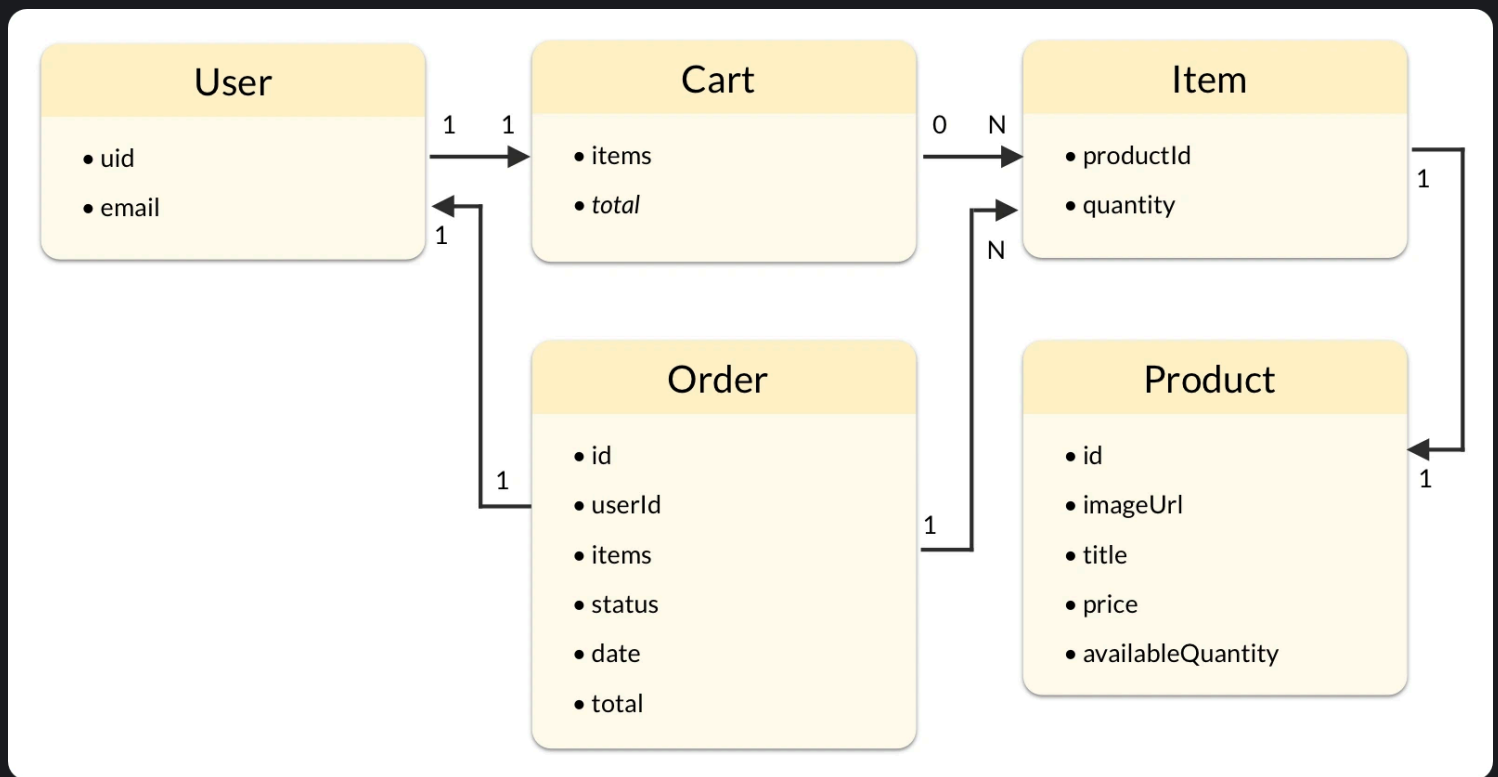
The domain model is a **conceptual model of the domain that incorporates both behavior and data**.

The **data** can be represented by a set of **entities** along with their **relationships**, while the **behavior** is encoded by some **business logic** for manipulating those entities.

Using an eCommerce application as an example, we could identify the following entities:

- **User:** ID and email
- **Product:** ID, image URL, title, price, available quantity etc.
- **Item:** Product ID and quantity
- **Cart:** List of items, total

- **Order:** List of items, price paid, status, payment details etc.



eCommerce app: entities and their relationships

When practicing DDD, entities and relationships are not something we produce out of thin air, but rather the end result of a (sometimes long) knowledge discovery process. As part of the process, a domain **vocabulary** is also formalized and used by all parties.

Note how at this stage we are not concerned about where these entities come from or how they are passed around in the system.

What's important is that our entities are at the **heart** of our system, because we need them to solve domain-related problems for our users.

In DDD, a distinction is often made between **entities** and **value objects**. For more info, see [this thread about Value vs Entity objects on StackOverflow](#).

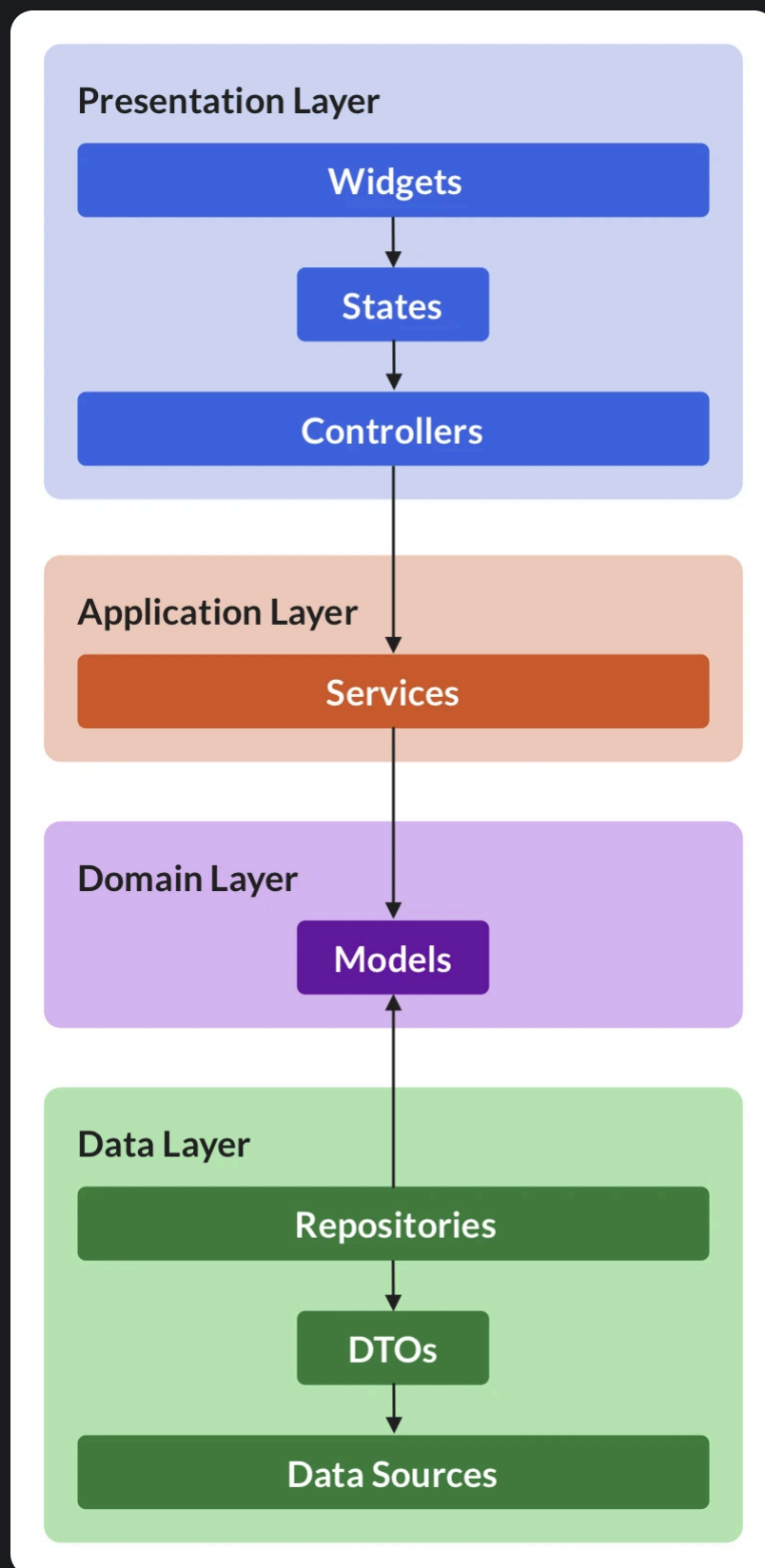
Of course, once we start building our app we need to implement those entities and decide where they fit within our architecture.

And this is where the domain layer comes in.

Going forward, we will refer to entities as **models** that can be implemented as simple classes in Dart.

The Domain Layer

Let's revisit our architecture diagram:



Flutter App Architecture using data, domain, application, and presentation layers.

As we can see, the models belong to the domain layer. They are retrieved by the repositories in the data layer below, and can be modified by the services in the application layer above.

So how do these models look like in Dart?

Well, let's consider a `Product` model class for example:

```
/// The ProductID is an important concept in our domain
/// so it deserves a type of its own
typedef ProductID = String;

class Product {
  Product({
    required this.id,
    required this.imageUrl,
    required this.title,
    required this.price,
    required this.availableQuantity,
  });

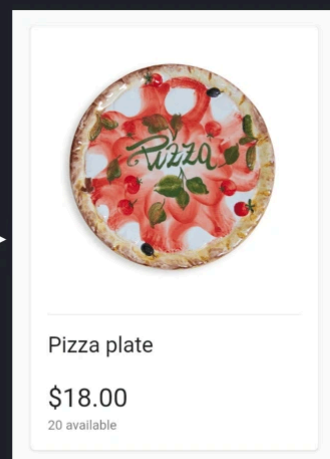
  final ProductID id;
  final String imageUrl;
  final String title;
  final double price;
  final int availableQuantity;

  // serialization code
  factory Product.fromMap(Map<String, dynamic> map, ProductID id) {
    ...
  }

  Map<String, dynamic> toMap() {
    ...
  }
}
```

At a very minimum, this class holds all the properties that we need to show in the UI:

```
class Product {
  Product({
    required this.id,
    required this.imageUrl,
    required this.title,
    required this.price,
    required this.availableQuantity,
  });
  final String id;
  final String imageUrl;
  final String title;
  final double price;
  final int availableQuantity;
}
```



And it also contains `fromMap()` and `toMap()` methods that are used for serialization.

There are various ways to define model classes and their serialization logic in Dart. For more info, see my [essential guide to JSON parsing in Dart](#) and the follow up article about [code generation using Freezed](#).

Note how the `Product` model is a **simple data classes** that doesn't have access to repositories, services, or other objects that belong **outside** the domain layer.

Business logic in the Model classes

Model classes can however include some business logic to express how they are meant to be modified.

To illustrate this, let's consider a `Cart` model class instead:

```
class Cart {  
  const Cart([this.items = const {}]);  
  /// All the items in the shopping cart, where:  
  /// - key: product ID  
  /// - value: quantity  
  final Map<ProductID, int> items;  
  
  factory Cart.fromMap(Map<String, dynamic> map) { ... }  
  Map<String, dynamic> toMap() { ... }  
}
```

This is implemented as a map of key-value pairs representing the product IDs and quantities of the items that we have added to the shopping cart.

And since we can add and remove items from the cart, it may be useful to define an **extension** that makes this task easier:

```
/// Helper extension used to update the items in the shopping cart.  
extension MutableCart on Cart {  
  Cart addItem({required ProductID productId, required int quantity}) {  
    final copy = Map<ProductID, int>.from(items);  
    // * update item quantity. Read this for more details:  
    // * https://codewithandrea.com/tips/dart-map-update-method/  
  }
```



```

    copy[productId] = quantity + (copy[productId] ?? 0);
    return Cart(copy);
  }

  Cart removeItemById(ProductID productId) {
    final copy = Map<ProductID, int>.from(items);
    copy.remove(productId);
    return Cart(copy);
  }
}

```

The methods above make a copy of the items in the cart (using `Map.from()`), modify the values inside, and return a new immutable `Cart` object that can be used to update the underlying data store (via the corresponding repository).

If you're not familiar with the syntax above, read: [How to update a Map of key-value pairs in Dart](#).

Many state management solutions rely on **immutable objects** in order to propagate state changes and ensure that our widgets rebuild only when they should. The rule is that when we need to mutate state in our models, we should do so by making a new, **immutable copy**.

Testing the Business Logic inside our Models

Note how the `Cart` class and its `MutableCart` extension don't have dependencies to any objects that live **outside** the domain layer.

This makes them very easy to test.

To prove this point, here's a set of unit tests that we can write to verify the logic in the `addItem()` method:

```

void main() {

  group('add item', () {

    test('empty cart - add item', () {
      final cart = const Cart()
        .addItem(productId: '1', quantity: 1);
      expect(cart.items, {'1': 1});
    });
  });
}

```

```

});

test('empty cart - add two items', () {
  final cart = const Cart()
    .addItem(productId: '1', quantity: 1)
    .addItem(productId: '2', quantity: 1);
  expect(cart.items, {
    '1': 1,
    '2': 1,
  });
});

test('empty cart - add same item twice', () {
  final cart = const Cart()
    .addItem(productId: '1', quantity: 1)
    .addItem(productId: '1', quantity: 1);
  expect(cart.items, {'1': 2});
});
}

```

Writing unit tests for our business logic is not only easy, but **adds a lot of value**.

If our business logic is incorrect, we are **guaranteed** to have bugs in our app. So we have every incentive to make it easy to test, by ensuring our model classes don't have any dependencies.

Conclusion

We have discussed the importance of having a good **mental model** of our system.

We've also seen how to represent our models/entities as **immutable** data classes in Dart, along with any business logic we may need to modify them.

And we've seen how to write some simple unit tests for that business logic, without resorting to mocks or any complex test setup.

Here are some tips you may use as you design and build your apps:

- explore the domain model and figure out what **concepts** and **behaviors** you need to represent
- express those concepts as **entities** along with their **relationships**

- implement the corresponding Dart model classes
- translate the behaviors into working code (business logic) that operates on those model classes
- add unit tests to verify the behaviors are implemented correctly

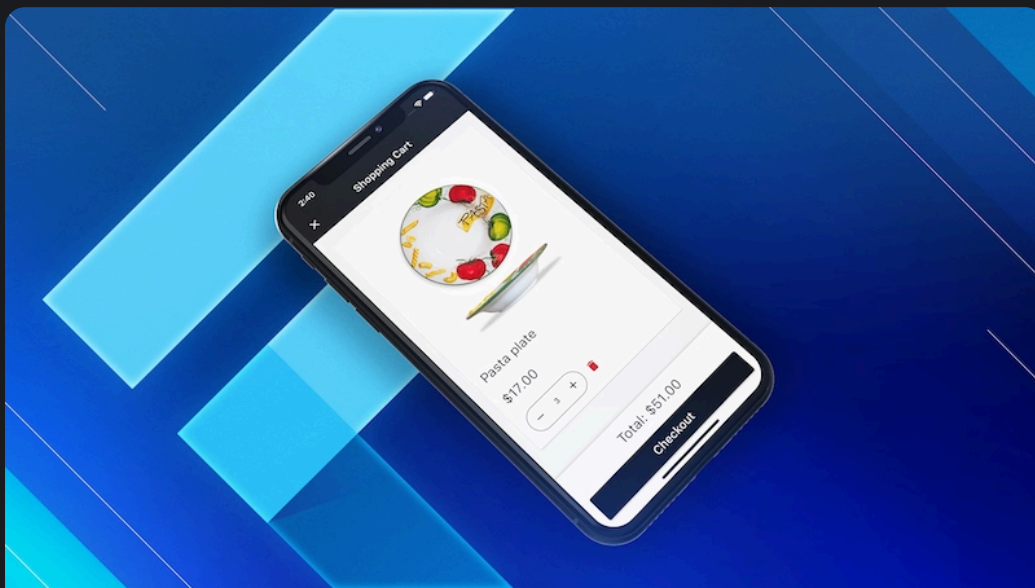
As you do that, think about what data you need to show in the UI and how the user will interact with it.

But don't worry about how things connect together just yet. In fact, it's the job of the **services** in the **application layer** to work with the models by mediating between the repositories in the data layer and the controllers in the presentation layer.

And that will be the subject of a future article.

Flutter Foundations Course Now Available

I launched a brand new course that covers Flutter app architecture in great depth, along with other important topics like state management, navigation & routing, testing, and much more:



Flutter Foundations Course

INTERMEDIATE TO ADVANCED

Learn about State Management, App Architecture, Navigation, Testing, and much more by building a Flutter eCommerce app on iOS, Android, and web.

Want More?

Invest in yourself with my high-quality Flutter courses.



Flutter In Production

INTERMEDIATE TO ADVANCED

Learn about flavors, environments, error monitoring, analytics, release management, CI/CD, and finally ship your Flutter apps to the stores. 🚀



Flutter & Firebase Masterclass

INTERMEDIATE TO ADVANCED

Learn about Firebase Auth, Cloud Firestore, Cloud Functions, Stripe payments, and much more by building a full-stack eCommerce app with Flutter & Firebase.



The Complete Dart Developer Guide

BEGINNER

Learn Dart Programming in depth. Includes: basic to advanced topics, exercises, and projects. Last updated to Dart 2.15.



Flutter Animations Masterclass

INTERMEDIATE

Master Flutter animations and build a completely custom habit tracking application.

[Contact](#)

[Twitter](#)

[Discord](#)

[GitHub](#)

[RSS](#)

[Meta](#)

[Privacy Policy](#)

[Terms of Use](#)